User CL Simulation Testing Setup

This setup is very similar to the tutorial on setting up a local environment for development documented in our documentation.

As such you can lookup some of the needed requirements there.

0. Requirements

- have an LDAP mafiasi account for access to the CLs

1. Setup and download our software

- SSH into the `cl0*` with your mafiasi user
- **Add your SSH key to GitHub to access and sync our repositories**
    - If you don't know what I am talking about or you don't yet have a SSH key, follow this guide: https://docs.github.com/en/authentication/connecting-to-github-with-ssh/checking-for-existing-ssh-keys
    - Go to your account settings and add your SSH key (the `.pub` file) to GitHub
- setup bitbots_main in your home directory

```
mkdir -p "$HOME/colcon_ws/src"
cd "$HOME/colcon_ws/src"
git clone git@github.com:bit-bots/bitbots_main.git && cd bitbots_main
make install-no-root
```

- set PATH and COLCON_WS (see section 5)

2. Compile the packages

If while testing you are changing code or updating `bitbots_main` via `make pull-all`, this step needs to be done again. For compilation of the whole meta repository run `cba`, which is an alias for: `cd $COLCON_WS; colcon build --symlink-install --continue-on-error` After a successful run, before we are able to use any ros commands we now need to source colcon built sources with `sa`, which is an alias for: `source "/opt/ros/jazzy/setup.$SHELL" && source "$COLCON_WS/install/`

`setup.$SHELL"`

3. Run Webots Simulation

We can start the Webots simulator with the following command: `rl bitbots_bringup simulator_teamplayer.launch game_controller:=false` This should start the simulation environment in the Webots simulator, while also starting all necessary nodes of the robot software (walking, vision, etc.). In the simulator we should see a field with a single robot. `rl` is short for `ros2 launch` and can be used as an alias as described in section 5.

With `game_controller:=false` we ensure, that the game_controller_listener is not started as well, but instead we will simulate the current gamestate by our own script (in another terminal): `rr game_controller_hl sim_gamestate.py`

Which allows us to simulate the current gamestate and different phases of the game. Now everything is ready for some simulation testing.

4. Testing and Debugging

By changing the simulated gamestate and seeing how the robot reacts, we can test our behavior. If there are issues with the robots behavior, they most likely have to do with DSD configuration or different parallelism handling in ROS 2. To be able to visualize the current DSD execution, we can start `rqt` and in the `Plugins` menu select `RoboCup -> DSD-Visualization`. This will show us the current DSD execution and can help in finding deadlocks or behavior execution logic issues.

TODO: extend this section